

ESCUELA POLITÉCNICA NACIONAL

FACULTAD DE INGENIERÍA DE SISTEMAS

APLICATIVO MÓVIL DE SEGURIDAD PREVENTIVA ALERTIFY

MÓDULO DE RUTEO SEGURO

TRABAJO DE INTEGRACIÓN CURRICULAR PRESENTADO COMO REQUISITO PARA LA OBTENCIÓN DEL TÍTULO DE INGENIERO EN SOFTWARE

ERICK SEBASTIÁN BALLAS PAGUAY

erick.ballas@epn.edu.ec

DIRECTOR: VÍCTOR VICENTE VELEPUCHA BONETT

victor.velepucha@epn.edu.ec

DMQ, febrero 2026

CERTIFICACIÓN

Yo, ERICK SEBASTIÁN BALLAS PAGUAY declaro que el trabajo de integración curricular aquí descrito es de mi autoría; que no ha sido previamente presentado para ningún grado o calificación profesional; y, que he consultado las referencias bibliográficas que se incluyen en este documento.

(ERICK SEBASTIÁN BALLAS PAGUAY)

Certifico que el presente trabajo de integración curricular fue desarrollado por ERICK SEBASTIÁN BALLAS PAGUAY, bajo mi supervisión.

(VÍCTOR VICENTE VELEPUCHA BONETT)
DIRECTOR

DECLARACIÓN DE AUTORÍA

A través de la presente declaración, afirmamos que el trabajo de integración curricular aquí descrito, así como el (los) producto(s) resultante(s) del mismo, son públicos y estarán a disposición de la comunidad a través del repositorio institucional de la Escuela Politécnica Nacional; sin embargo, la titularidad de los derechos patrimoniales nos corresponde a los autores que hemos contribuido en el desarrollo del presente trabajo; observando para el efecto las disposiciones establecidas por el órgano competente en propiedad intelectual, la normativa interna y demás normas.

ERICK SEBASTIÁN BALLAS PAGUAY

VÍCTOR VICENTE VELEPUCHA BONETT

JOHAN STEVEN BAÑO AREQUIPA

LUIS ANIBAL ROCHA ROCHA

DEDICATORIA

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

AGRADECIMIENTOS

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Índice general

1. INTRODUCCIÓN	1
1.1. Objetivo general	2
1.2. Objetivos específicos	2
1.3. Alcance	3
1.4. Marco teórico	3
1.4.1. Percepción de la Inseguridad	4
1.4.2. Metodología tradicional con Modelo Cascada	5
1.4.3. Metodologías Alternativas de Desarrollo de Software	6
1.4.4. Metodología Desarrollo Ágil de Software	6
1.4.5. Arquitectura de Software	10
1.4.6. Herramientas y Tecnologías	13
2. METODOLOGÍA	20
2.1. Fase Exploratoria	20
2.1.1. Stakeholders	20
2.1.2. Planificación del proyecto	21
2.2. Fase Exploratoria	23
2.2.1. Herramientas y Entorno de Trabajo	23
2.2.2. Definición de requerimientos	24
2.2.3. Definición de Sprints	32
2.2.4. Sprint 0	33
3. RESULTADOS, CONCLUSIONES Y RECOMENDACIONES	38
3.1. Resultados	38
3.2. Conclusiones	39
3.3. Recomendaciones	39

4. REFERENCIAS BIBLIOGRÁFICAS	40
I. Conjunto de Datos Extensos	46
II. Formato de Entrevista	47
III. Enlaces	48

Índice de figuras

1.1. Modelo Cascada [9]	5
1.2. Estructura Scrum [16]	8
1.3. Arquitectura de Microservicios	13
2.1. Cronograma de actividades del proyecto	22
2.2. Arquitectura de Microservicios del Módulo de Ruteo Seguro en Alertify	34
2.3. Prototipos de la aplicación móvil para el Módulo de Ruteo Seguro	34
2.4. Esquema de la base de datos	35
2.5. Organización en GitHub para el proyecto Alertify	36
2.6. Tablero de gestión en Jira para el Módulo de Ruteo Seguro	37
3.1. Resultados de las pruebas realizadas	39

Índice de Tablas

2.1. Partes Interesadas (Stakeholders)	21
2.2. Roles y responsabilidades del marco Scrum en el proyecto.	23
2.3. Herramientas y tecnologías utilizadas en el desarrollo del sistema.	24
2.4. Definición de épicas y sus historias de usuario asociadas	25
2.5. Historia de Usuario HU-01: Selección de Origen y Destino.	26
2.6. Historia de Usuario HU-02: Cálculo de Ruta Segura.	26
2.7. Historia de Usuario HU-03: Visualización de Trayecto.	27
2.8. Historia de Usuario HU-04: Ponderación de Riesgo.	27
2.9. Historia de Usuario HU-05: Recálculo de Ruta por Desvío.	28
2.10. Historia de Usuario HU-06: Carga y Estructuración de Red Vial.	28
2.11. Escala de Estimación de Complejidad para Historias de Usuario	29
2.12. Escala de Priorización para Historias de Usuario	29
2.13. Desglose de tareas por Historia de Usuario	30
2.14. Desglose de tareas por Sprint	32
3.1. Resultados de las pruebas realizadas	38

RESUMEN

(Máximo 250 palabras. Colocar al final palabras clave, hasta seis)

Palabras Claves - español, test 1

ABSTRACT

(Resumen en inglés. Máximo 250 palabras. Colocar al final Keywords)

Keywords - english, test 2

Capítulo 1

INTRODUCCIÓN

Dentro del contexto actual, la alta densidad poblacional y el crecimiento significativo de la tasa de criminalidad son desafíos constantes que se presentan en ciudades grandes, como Quito. Esta realidad impacta directamente en la calidad de vida e impide el desarrollo normal de las actividades, poniendo en cuestión este tema. Uno de los factores más afectados por la inseguridad es la movilidad. Esta problemática genera un ambiente de incertidumbre sobre para la seguridad vial y personal, en donde la planificación de rutas seguras no solo se basa en la distancia sino también en la variable crítica de seguridad durante el trayecto. En este ámbito, el surgimiento de herramientas tecnológicas accesibles e innovadoras es fundamental para que se permita a la ciudadanía tomar decisiones informadas y cautelares durante sus desplazamientos cotidianos.

Como respuesta a esta necesidad, se asumió el desarrollo de un aplicativo móvil el cual brinde una herramienta de seguridad preventiva, en donde su enfoque principal son usuarios jóvenes y adultos, mediante la cual se facilite la forma en la que se desplazan por la ciudad; estos mismos brindan la información en tiempo real sobre zonas con un alto potencial de ser peligrosas.

De esta manera, la aplicación ha sido concebida con una estructura autónoma y modular, que se divide en tres componentes principales: Gestión de Identidad y Autenticación, Gestión de Reportes y Alertas y Ruteo Seguro; a primera instancia este documento se centra en la implementación del Módulo de Ruteo Seguro el cual tiene como propósito fundamental la generación de rutas óptimas para el usuario.

Este componente maneja la operatividad de la aplicación basándose en el análisis de incidentes previamente reportados y validados por la comunidad, para brindar mayor fiabilidad en la sugerencia de rutas; con la ayuda de principios de algoritmos y optimización de grafos.

De modo que este módulo está diseñado para brindar un camino seguro evaluando el factor de seguridad como el valor dominante para el peso en la selección de la ruta, este mismo garantiza la independencia del servicio al hacer uso de una API robusta la cual puede calcular y devolver una ruta adecuada, asegurando la comunicación eficiente y manteniendo una disponibilidad continua en la aplicación.

En cuanto, al Módulo de Gestión de Identidad y Autenticación es el aquel que constituye la entrada al sistema y es el pilar fundamental de la seguridad. Dentro de sus funciones principales incluye el inicio de sesión y la gestión de la sesión activa para mantener al usuario dentro del aplicativo. Es un módulo el cual se maneja por Json Web Tokens, el funcionamiento de este módulo es crítico para mantener una sinergia con el Módulo de Gestión de Reportes y Alertas, en donde este tercer y último módulo juega un papel vital para la funcionalidad de seguridad preventiva; su rol principal consiste en validar y procesar los reportes de los incidentes ingresados, en donde no solo almacena la información del módulo sino también las coordenadas de los incidentes validando su veracidad de manera que este sirva para generar un mapa de riesgo fiable. Además es el responsable de emisión de notificaciones a los usuarios que se encuentren dentro de una zona de riesgo.

1.1. Objetivo general

Elaborar el componente de ruteo seguro que genere caminos óptimos y preventivos, calculando las rutas con base en el mínimo riesgo.

1.2. Objetivos específicos

1. Establecer los requisitos funcionales y no funcionales detallados para el cálculo y visualización de rutas seguras.
2. Formalizar la planificación del desarrollo del módulo mediante la creación de Historias de Usuario.
3. Desarrollar la estructura de la base de datos con capacidad espacial y asegurar la integración de los datos de peligrosidad para el cálculo de rutas.
4. Desarrollar la interfaz de usuario del Módulo de Ruteo para que los usuarios puedan solicitar y visualizar de manera intuitiva las rutas seguras propuestas, facilitando la toma de decisiones informadas antes de iniciar sus desplazamientos.

5. Desarrollar la API del Servicio de Ruteo, permitiendo la solicitud y la entrega de rutas seguras en tiempo real y garantizando la interoperabilidad con los módulos de Reportes y Autenticación.
6. Realizar pruebas funcionales y de usabilidad para comprobar la precisión del algoritmo en la generación de rutas de bajo riesgo.

1.3. Alcance

El presente trabajo se centra en el diseño, desarrollo e implementación del Módulo de Ruteo Seguro, el mismo que funcionará como un microservicio autónomo dentro de la arquitectura de Alertify. En donde el esfuerzo principal se centra en la creación de un Motor de Ruteo que debe ser capaz de consumir datos geográficos de peligrosidad provenientes del Módulo de Reportes, de manera que a través de un algoritmo se pondere la minimización del riesgo sobre la distancia, siendo este el entregable funcional central. A su vez, la implementación de la Base de Datos de Red Vial con capacidad para almacenar y gestionar eficientemente las estructuras de datos espaciales (nodos y caminos) requeridas por el motor de cálculo asegurando así una comunicación eficiente entre el aplicativo móvil, dentro del mismo efecto se desarrollará la API y respectivo frontend los mismos que exprondran la operatividad del motor de ruteo, asegurando que su puesta en marcha cumpla con las correspondientes pruebas funcionales y no funcionales.

1.4. Marco teórico

La idea de que la tecnología tiene que ser un instrumento para prevenir riesgos es la base de este módulo. El concepto principal es incluir la información comunitaria acerca de la inseguridad urbana en el procedimiento de cálculo de rutas. Esto quiere decir que el sistema emplea los datos de incidentes para otorgar un peso o riesgo a las calles. En donde el objetivo principal es convertir los datos de peligrosidad en un elemento de navegación, garantizando que el usuario tenga la opción de elegir una ruta que lo exponga al riesgo en la ciudad lo menos posible.

1.4.1. Percepción de la Inseguridad

La seguridad ciudadana es un pilar esencial para el desarrollo y el bienestar de cualquier sociedad. En Ecuador, no basta con pensar que la seguridad es solo que no haya crímenes, sino que es algo mucho más intrincado y con un contexto que va más allá. Estudios recientes señalan que la falta de seguridad está muy ligada a temas sociales, económicos y a si la gente confía o no en las entidades del gobierno [1]. Esta conexión tiene un efecto que va más allá de cuántos delitos hay, pues afecta directamente cómo vive la gente.

Algo muy importante aquí es cómo la gente siente la inseguridad. Los estudios sobre ser víctima de un delito en Ecuador muestran que el temor a que te roben o te hagan daño puede ser tan malo como que te pase de verdad, lo cual cambia el tipo de rutina diaria, te impide moverte libremente y hace que la sociedad se debilite [2]. Es en esta situación donde la gente siente mucho riesgo y se hace evidente la necesidad de avanzar de modelos de seguridad puramente reactivos hacia enfoques más proactivos y preventivos.

El Rol de la Tecnología

Ante las pruebas que presenta la seguridad en la ciudad, las tácticas actuales han empezado a dar más importancia a evitar los delitos. En esta idea, la ciudadanía toma el papel principal para adelantarse a los problemas, en vez de solo reaccionar a ellos [3]. En este cambio, las herramientas digitales se han vuelto muy importantes, dando opciones nuevas para manejar la seguridad [4]. Que casi todos tengan celulares ha creado una oportunidad nueva y fuerte en este tema. Poder crear y mandar datos al instante ha hecho que la gente sea como un radar que siente lo que pasa a su alrededor. Un estudio detallado de textos dice que los datos de los celulares son muy valiosos para ver y saber cuándo habrá delitos, lo que da una base firme para usarlos para evitar el crimen [5]. El sitio más claro en donde se evidencia capacidad se puede observar en las apps móviles enfocadas en la seguridad de los ciudadanos. Hoy en día, estas plataformas son un medio para reportar y manejar eventos criminales, demostrando que operan de forma eficaz como instrumentos de vigilancia colectiva [6]. Un buen ejemplo es la app SOSAFE, en donde se evidencia cómo estas tecnologías pueden mejorar la gestión de la seguridad en las ciudades a nivel local [7]. Al hacer más fácil que los vecinos se comuniquen al instante entre ellos y con las autoridades, estas apps no solo aceleran la respuesta, sino que también promueven la unión y la responsabilidad de todos. El proyecto Alertify encaja de lleno en esta nueva forma de hacer las

cosas. Apuesta por la seguridad preventiva, dando a los usuarios información al momento creada por la comunidad para decidir mejor, por ejemplo, qué camino tomar. Para que esto funcione, es clave cuidar la seguridad, la privacidad y la protección de los datos del usuario, ya que la gente tiene que confiar en la plataforma para usarla y que sea útil [8]. Alertify, en resumen, no es solo una herramienta tecnológica, sino una forma de reforzar la sociedad y que la gente se sienta más segura en la ciudad.

1.4.2. Metodología tradicional con Modelo Cascada

La creación de software cuenta con varios modelos de desarrollo, cada uno con ventajas específicas para diferentes tipos de proyectos. El enfoque tradicional por excelencia es el Modelo de Cascada (Waterfall). Su mayor ventaja es su organización secuencial y lineal, la cual brinda transparencia, predecibilidad y una gestión fácil de proyectos al requerir que cada etapa (análisis, diseño, implementación, pruebas, mantenimiento) termine antes de comenzar la siguiente [9]. Este modelo es muy eficaz para proyectos en los que los requisitos son estables, se entienden adecuadamente desde el comienzo y es fundamental crear una documentación técnica detallada durante el proceso [10]. Esta metodología se la evidencia en la Figura 1.1.

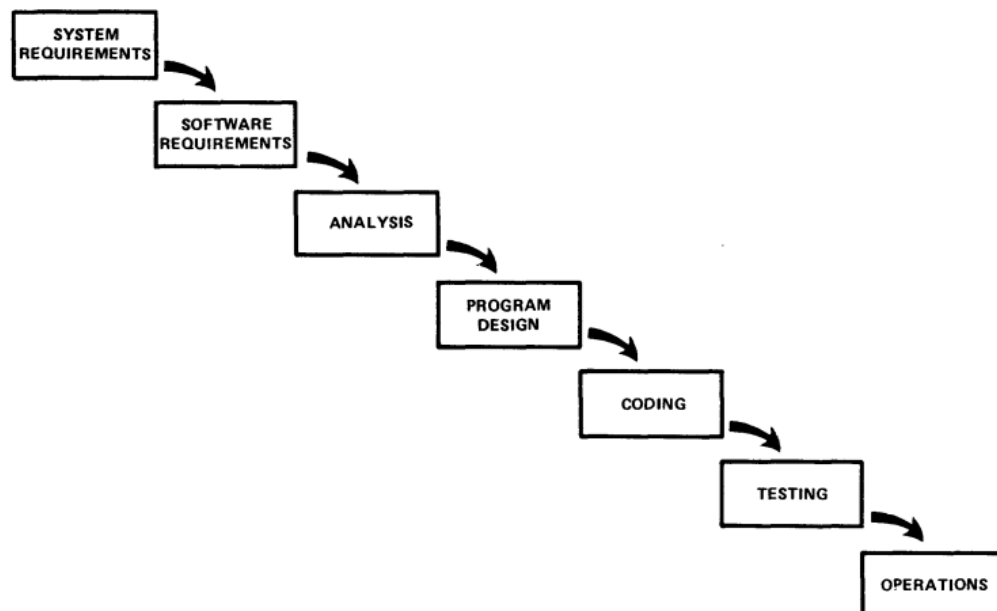


Figura 1.1: Modelo Cascada [9]

1.4.3. Metodologías Alternativas de Desarrollo de Software

Así mismo existen otros modelos que ofrecen beneficios para contextos específicos y se muestra que enfoques ágiles y lineales no son la única forma de desarrollar software.

Modelo Espiral

La gestión y la reducción de riesgos fueron el enfoque para el que se desarrolló el Modelo Espiral [11]. Su mayor beneficio es que es de naturaleza iterativa, lo que significa que cada ciclo (o espiral) contiene un análisis de riesgo explícito. Esto lo hace apropiado para proyectos que son grandes, complejos y técnicamente peligrosos [11].

Modelo-V

El Modelo-V, por su parte, es un modelo que ha evolucionado a partir del modelo Cascada y tiene como principal ventaja un proceso meticuloso de verificación y validación [12]. Su estructura conecta cada fase de desarrollo con una fase de prueba correspondiente, lo que asegura un alto nivel de calidad y disciplina, algo esencial en sistemas donde los errores son críticos [12].

Modelo Prototipado

El modelo de prototipado, además, se enfoca en el beneficio de aclarar requisitos; se emplea cuando los requerimientos del usuario no son claros y permite crear rápidamente un prototipo para recibir comentarios tempranos antes de desarrollar el sistema final [13].

1.4.4. Metodología Desarrollo Ágil de Software

En proyectos como Alertify, donde las necesidades de los usuarios y las condiciones del entorno, en este caso la seguridad en la ciudad, cambia mucho no se debe ser demasiado rígido ya que puede resultar en un producto final que no satisfaga al usuario o que ya no sirva para cuando salga al mercado. Para abordar entornos donde los requisitos son inherentemente volátiles o no se comprenden completamente al inicio, surgieron las metodologías Ágiles. El beneficio clave de los enfoques ágiles, en contraposición a los modelos rígidos, es su capacidad intrínseca de adaptación al cambio [14]. En vez de planear un lanzamiento final grande, estos modelos fomentan la entrega continua e incremental de

software que es funcional. Su utilidad se enfoca en la adaptabilidad, el trabajo colaborativo constante con los clientes (o sus representantes) y la habilidad de re-priorizar las tareas en función de un feedback continuo[10]. Este método ayuda a mejorar la capacidad de predecir y a mantener el riesgo bajo control en situaciones que se pueden salir de las manos, siendo perfecto para proyectos difíciles donde los requisitos pueden cambiar [15].

Scrum como Marco de Trabajo

En el ámbito ágil, Scrum destaca como uno de los modelos de trabajo más extendidos y preferidos [16]. Se concibe como un esquema de procesos que usa una estrategia repetitiva y progresiva para direccionar la creación de productos intrincados [15]. Su ejecución se cimienta en una doctrina de mando de procedimientos práctica, que se apoya en tres fundamentos clave: claridad, revisión y ajuste [15].

En lo que respecta al proyecto Alertify, la selección de Scrum se fundamenta en su aptitud para brindar un avance dinámico, cooperativo y productivo, sobre todo en ambientes intrincados como el de la protección de datos [17]. Seguidamente, se exponen los elementos de este modelo de trabajo y su trascendencia para el proyecto en cuestión.

La Estructura de Scrum

La implementación de Scrum se organiza en torno a un conjunto de roles, eventos y artefactos que trabajan en sinergia para entregar valor de manera continua [16]. En la Figura 1.2 se ilustra la estructura de Scrum y sus componentes principales.

Roles en Scrum (El Equipo Scrum)

El equipo es autoorganizado y multifuncional, lo que garantiza que el trabajo se realice sin dependencias externas [15].

- **Product Owner (Dueño del Producto):** Es el único responsable de gestionar el *Product Backlog* (la lista de requisitos del producto) y de maximizar el valor del trabajo realizado por el equipo de desarrollo. Actúa como la voz del cliente y los stakeholders, asegurando que el equipo construya el producto correcto [16]. Para Alertify, es quien traduce las necesidades de seguridad de los usuarios en funcionalidades concretas y priorizadas.

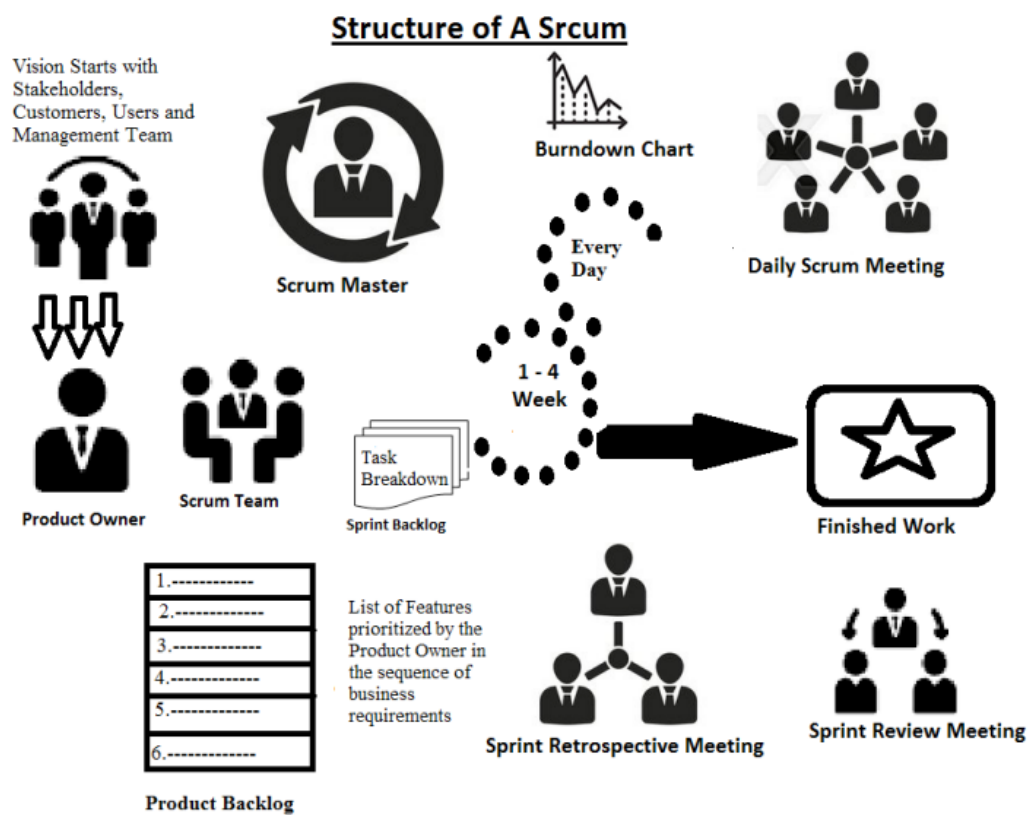


Figura 1.2: Estructura Scrum [16]

- **Development Team (Equipo de Desarrollo):** Está compuesto por los profesionales (desarrolladores, testers, diseñadores, etc.) que se encargan de entregar un incremento de producto "Terminadoz potencialmente desplegable en cada Sprint. Nadie, ni siquiera el Scrum Master, les indica cómo convertir los elementos del *Product Backlog* en funcionalidades [15].
- **Scrum Master:** Actúa como un líder servicial para el equipo. Su función principal es asegurar que Scrum sea entendido y adoptado, eliminando impedimentos y facilitando los eventos de Scrum según sea necesario [16]. Es un coach que protege al equipo de interrupciones externas y guía a la organización en la adopción de Scrum [16].

Eventos en Scrum

Estos eventos están diseñados para minimizar la necesidad de reuniones no planificadas y crear una cadencia regular de inspección y adaptación [15].

- **El Sprint:** Es el corazón de Scrum, un ciclo de trabajo de duración fija (generalmente entre una y cuatro semanas) en el que se crea un incremento de producto utilizable [16]. Un nuevo Sprint comienza inmediatamente después de la conclusión del anterior [15].
- **Sprint Planning:** Al inicio de cada Sprint, todo el equipo Scrum colabora para definir qué se puede entregar en el siguiente incremento y cómo se logrará ese trabajo [15].
- **Daily Scrum:** Es una reunión diaria de 15 minutos para el Equipo de Desarrollo. Su propósito es sincronizar actividades y crear un plan para las próximas 24 horas, inspeccionando el progreso hacia el objetivo del Sprint [15].
- **Sprint Review:** Al final del Sprint, el equipo y los stakeholders inspeccionan el incremento y adaptan el *Product Backlog* si es necesario. Es una sesión para demostrar el trabajo "Terminadoz obtener retroalimentación [16].
- **Sprint Retrospective:** Es una oportunidad para que el equipo se inspeccione a sí mismo y cree un plan de mejoras para el próximo Sprint, enfocándose en sus procesos, relaciones y herramientas [15].

Artefactos de Scrum

Representan el trabajo o valor y están diseñados para maximizar la transparencia de la información clave [15].

- **Product Backlog:** Es la única fuente de requisitos del producto. Se trata de una lista ordenada y dinámica de todo lo que se conoce que es necesario para el producto (características, funciones, mejoras, correcciones), gestionada por el Product Owner [16].
- **Sprint Backlog:** Es el conjunto de elementos del *Product Backlog* seleccionados para un Sprint, más el plan para entregarlos y alcanzar el Objetivo del Sprint (Sprint Goal) [15].
- **Incremento:** Es la suma de todos los elementos del Product Backlog completados durante un Sprint y todos los Sprints anteriores. Al final de cada Sprint, el nuevo incremento debe estar "Terminado", lo que significa que está en condiciones de ser utilizado [15].

1.4.5. Arquitectura de Software

Una arquitectura de software apropiada es la estructura principal que tiene un impacto directo en la habilidad del sistema para evolucionar, mantenerse actualizado y adaptarse a lo largo del tiempo. Cuando se trata de una herramienta flexible como Alertify, es crucial optar por un modelo que posibilite que cada componente funcione de manera independiente y, además, facilite su desarrollo en el futuro.

Tipos de Arquitectura

Existen varios tipos de arquitecturas de software, cada con sus respectivas ventajas y características que las diferencian entre sí.

Arquitectura Monolítica

La arquitectura monolítica representa el enfoque tradicional donde una aplicación se construye como una única unidad. En este modelo, todos los componentes funcionales están fuertemente acoplados, comparten una sola base de código y se despliegan como un

solo artefacto [18]. La aplicación se basa en un solo proceso o contenedor, que administra las distintas labores (como la autenticación, el ruteo o los reportes) dentro de una misma estructura de código.

Arquitectura Dirigida por Eventos

La Arquitectura Dirigida por Eventos (EDA) es un paradigma de diseño de software en el cual el flujo del sistema es determinado por eventos, que son notificaciones de cambios de estado significativos [19].

Arquitectura en Capas

La Arquitectura en Capas (Layered Architecture) es uno de los patrones más fundamentales y utilizados en el diseño de software, cuyo objetivo principal es la separación de intereses (separation of concerns) [20].

Arquitectura Orientada a Servicios (SOA)

La Arquitectura Orientada a Servicios (SOA) es un estilo arquitectónico que promueve la creación de aplicaciones a través de la orquestación de servicios de negocio reutilizables y débilmente acoplados [21].

Arquitectura Microservicios

La arquitectura de microservicios es un estilo arquitectónico que estructura una aplicación como una colección de servicios pequeños y autónomos [22]. Cada servicio se configura en función de una capacidad de negocio concreta, es administrado por un grupo independiente y tiene la posibilidad de ser implementado de manera autónoma [23]. Esta independencia posibilita que cada servicio funcione y crezca independientemente [18].

Beneficios de los Microservicios

Al hacer uso de una arquitectura de microservicios en Alertify, se justifica el hecho de operar un servicio de manera independiente y, de igual manera, brinda una serie de ventajas estratégicas que se alinean directamente con los objetivos y la estructura del proyecto:

- **Despliegue Independiente y Autonomía de Equipos:** Al dividir la aplicación en partes autónomas (Manejo de Reportes y Alertas, Manejo de Gestión de Identidad y Enrutamiento Seguro), cada integrante puede trabajar, testear y poner en marcha su parte de manera autónoma. Esto no solo hace que todo vaya más rápido, sino que también encaja perfectamente con la estructura del proyecto [23].
- **Escalabilidad Selectiva:** No todos los componentes de Alertify van a tener el mismo peso. Por ejemplo, el servicio de “Ruteo Seguro” podría necesitar mucha potencia de cálculo en las horas de más movimiento, mientras que el servicio de “Autenticación” podría ir más relajado. La arquitectura de microservicios permite ampliar cada servicio por separado, según lo que pida, aprovechando mejor los recursos y mejorando el rendimiento general sin tener que ampliar toda la aplicación [18].
- **Resiliencia y Aislamiento de Fallos:** En una configuración monolítica, un error en una parte puede tirar abajo toda la aplicación. Al usar microservicios, el sistema se vuelve más resistente. Si el servicio de “Gestión de Reportes” falla por un momento, los usuarios de Alertify todavía podrían iniciar sesión y calcular rutas seguras, ya que los demás servicios seguirían operando de forma independiente [22]. Esto es fundamental para una aplicación de seguridad, donde la disponibilidad es lo más importante.
- **Flexibilidad Tecnológica:** En sí, cada microservicio puede ser creado utilizando un conjunto de tecnologías diferente. Aunque en Alertify todos los servicios de backend usarán NestJS, esta cualidad de la arquitectura facilita futuras actualizaciones, permitiendo adoptar la mejor herramienta para un problema específico sin estar atado a las decisiones tecnológicas del pasado [23].

Funcionamiento y Patrones de Implementación

En una arquitectura de microservicios, la comunicación es un factor clave. En vez de invocar funciones en un proceso específico, los servicios se comunican entre sí por medio de la red, usualmente a través de APIs RESTful [24]. Cada uno de los tres módulos de Alertify (Ruteo, Reportes y Autenticación) mostrará una serie de endpoints que pueden ser utilizados tanto por el cliente móvil, como por otros servicios.

Con el fin de administrar eficazmente esta comunicación y hacer más sencilla la lógica del lado del cliente, se aplicará el patrón API Gateway. Este funciona como un punto de acceso único para todas las peticiones que vienen del aplicativo móvil [25]. En vez de que

la aplicación tenga que saber las direcciones de red de los tres microservicios, simplemente se envía todas sus solicitudes al Gateway. Este mismo se encarga de dirigir cada petición al microservicio que le corresponde, añadir respuestas cuando sea necesario y administrar tareas transversales como la autenticación de tokens y el registro [25]. Esto permite desblindar al cliente de la estructura interna del backend, ofreciendo una interfaz sólida y limpia que hace más fácil el mantenimiento y desarrollo del ecosistema de servicios. En la Figura 1.3 se ilustra cómo se estructura la arquitectura de microservicios.

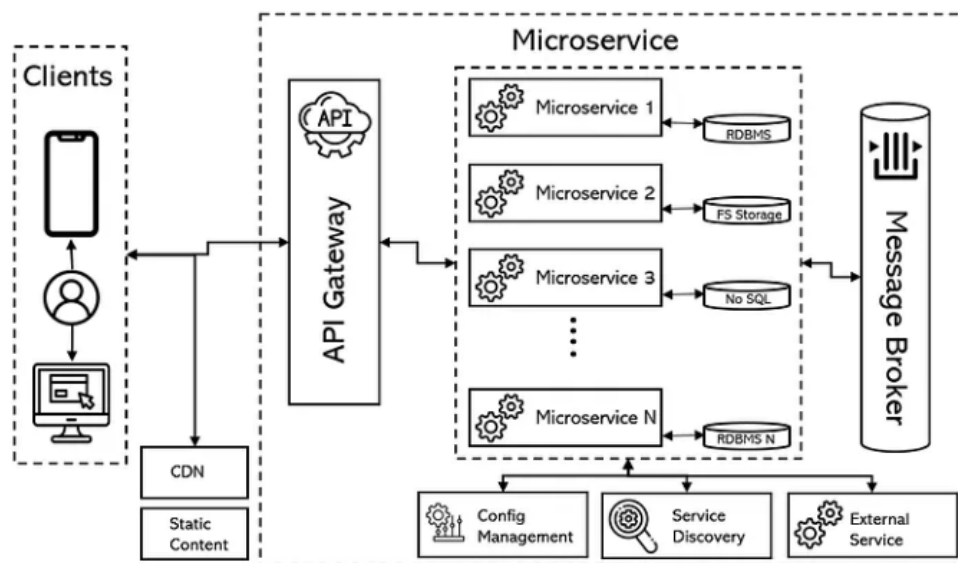


Figura 1.3: Arquitectura de Microservicios

[26]

1.4.6. Herramientas y Tecnologías

Con el objetivo de optimizar la productividad, garantizar que el sistema final sea robusto y asegurar su capacidad para escalar, cada uno de los elementos del stack tecnológico ha sido seleccionado con un enfoque estratégico.

Herramientas de diseño y prototipado

Figma

Figma es un instrumento de diseño moderna que se emplea para definir la experiencia del usuario (UX) y elaborar interfaces de usuario (UI) [27]. Se caracteriza por ser una pla-

taforma colaborativa en línea, lo que posibilita a los equipos de desarrollo y diseño trabajar juntos en tiempo real sobre las maquetas y prototipos de la aplicación. Para determinar el aspecto visual y la secuencia de interacción de la aplicación móvil antes de la etapa de implementación, su atención en el diseño UI/UX es esencial [27].

NinjaMock

NinjaMock se utiliza como una herramienta de prototipado rápido (wireframing). Su utilidad radica en permitir a los diseñadores y desarrolladores crear representaciones visuales de aplicaciones móviles y web de manera intuitiva y sin necesidad de codificación previa. Esta herramienta facilita la validación temprana de la navegación y el diseño de la aplicación, siendo un recurso valioso para conceptualizar aplicaciones móviles multiplataforma antes de iniciar la fase de implementación [28].

Jira

Jira se ha establecido como una herramienta fundamental en la gestión de proyectos de desarrollo de software, especialmente bajo metodologías ágiles. Su funcionalidad principal abarca la planificación, el rastreo y el lanzamiento de aplicaciones, permitiendo a los equipos de desarrollo organizar su trabajo de manera eficiente mediante tableros y flujos de trabajo personalizados. La capacidad de Jira para gestionar incidencias y visualizar el progreso del proyecto es esencial para asegurar la entrega de software de calidad y cumplir con los cronogramas establecidos [29].

Herramientas de Frontend

Android

Android es un sistema operativo móvil de código abierto y se fundamenta en una variante alterada del kernel de Linux [30]. Su diseño está orientado a dispositivos móviles con pantalla táctil, como las tabletas y los teléfonos inteligentes, y su estructura abierta ha promovido la adopción y personalización extensiva de parte de los fabricantes. Se ha consolidado como una de las plataformas más empleadas en el mundo entero, resaltando por su flexibilidad y su extenso ecosistema de aplicaciones [30].

El desarrollo de aplicaciones nativas para esta plataforma se lleva a cabo en su mayoría por medio del entorno de desarrollo integrado (IDE) oficial, Android Studio [31]. Este ambiente ofrece un paquete integral de herramientas especializadas que ayudan al desarrollador, incluyendo un editor de código, herramientas de depuración avanzadas y emuladores para evaluar la aplicación en una variedad de dispositivos virtuales [31].

Kotlin

Para crear el aplicativo móvil se eligió Kotlin como lenguaje de programación, ya que permite un desarrollo nativo en Android. Esta resolución se fundamenta en los importantes beneficios que Kotlin proporciona en comparación con opciones más convencionales como Java. Los estudios comparativos muestran que Kotlin posibilita redactar código con mayor concisión y expresividad, disminuyendo así la cantidad de código repetido (boilerplate) y, por lo tanto, el riesgo de errores [32].

Una de las cualidades más sobresalientes de Kotlin es su sistema de seguridad frente a nulos (null safety), que está incorporado directamente en el lenguaje. Este sistema hace una distinción entre referencias nulas y no nulas, eliminando durante la compilación los riesgosos errores de NullPointerException, que son una de las causas más frecuentes de fallos en las aplicaciones Android [32]. Kotlin también es completamente interoperable con Java, lo que quiere decir que puede convivir en un mismo proyecto sin inconvenientes y beneficiarse del amplio ecosistema de librerías y frameworks existentes en Java [33].

Herramientas de Backend

NestJS

Alertify empleará NestJS como framework de desarrollo y su backend se edificará empleando una arquitectura de microservicios. NestJS es un marco de trabajo para Node.js que simplifica la elaboración de aplicaciones del lado del servidor que sean confiables, escalables y eficaces. Su mayor beneficio es que establece una estructura modular y ordenada, inspirada en Angular, la cual es perfecta para implementar microservicios [34]. NestJS emplea TypeScript y promueve patrones de diseño robustos, como la Inyección de Dependencias; esto permite que el código esté más desacoplado, sea más fácil de mantener y, esencialmente, más sencillo de probar. Esta estructura es esencial para un sistema como Alertify, que está dividido en servicios independientes (Ruteo, Reportes y Autenticación).

Las evaluaciones de rendimiento en ambientes de nube contemporáneos han probado que Nest.JS es muy apropiado para aplicaciones con alta demanda, mostrando una eficiencia y escalabilidad sobresalientes [35].

Bases de Datos Relacionales (SQL) y No Relacionales (NoSQL)

Los enfoques principales que componen el panorama de las bases de datos son dos: NoSQL y SQL. Las bases de datos relacionales (SQL) tienen como objetivo asegurar la integridad de las transacciones a través de las propiedades ACID (Atomicidad, Aislamiento, Consistencia y Durabilidad) y distribuyen los datos en tablas estructuradas [36]. En cambio, los modelos que ofrecen las bases de datos NoSQL (como el documental o clave-valor) están diseñados para tener una alta escalabilidad y para gestionar datos no estructurados, poniendo a menudo la disponibilidad por encima de la consistencia estricta de ACID [36].

SQL Server

Para Alertify, que trabaja con datos estructurados y críticos como reportes geo-localizados, usuarios e información de carreteras, la opción más robusta es un modelo relacional. SQL Server fue seleccionado específicamente porque garantiza las propiedades ACID [36], lo que es esencial para evitar inconsistencias, como un reporte sin un usuario conectado. También, SQL Server proporciona un rendimiento mejorado para las consultas complejas que necesitan realizar operaciones de unión (joins) entre las tablas del sistema. La implementación de buenas prácticas de mantenimiento y administración garantiza que SQL Server funcione con seguridad y eficacia en contextos contemporáneos [37].

PostgreSQL

PostgreSQL representa una evolución significativa en los sistemas de gestión de bases de datos, introduciendo un enfoque objeto-relacional robusto. Su diseño arquitectónico permite extender el modelo de datos tradicional mediante la definición de nuevos tipos de datos, operadores y métodos de acceso. Esta capacidad de extensibilidad, combinada con un sistema de reglas activo, posiciona a PostgreSQL como una solución potente para manejar datos complejos y garantizar la integridad de la información en entornos que requieren un rigor transaccional estricto [38].

MongoDB

Como alternativa a los modelos relacionales, MongoDB ofrece una solución de almacenamiento de datos potente y altamente escalable, orientada a documentos. Esta tecnología permite gestionar información de manera flexible, sin la rigidez de los esquemas de tablas tradicionales, lo que resulta ideal para aplicaciones modernas que manejan grandes volúmenes de datos diversos. La arquitectura de MongoDB está diseñada para facilitar el escalado horizontal y proporcionar un alto rendimiento en la recuperación y almacenamiento de información [39].

Herramientas de Control de Versiones y Colaboración

Git

Git es un sistema de control de versiones distribuido (DVCS), el cual es una herramienta fundamental para la creación de software cooperativo [40]. Su objetivo principal es rastrear de manera eficaz las modificaciones que se han hecho al código fuente con el tiempo. Contrario a los sistemas centralizados, Git otorga a cada uno de los desarrolladores un repositorio local completo. Esto hace posible la creación de ramas (branching) y la fusión de código (merging) con una velocidad enorme y unas capacidades muy potentes [40].

Github

GitHub es una plataforma de alojamiento web que emplea Git como su motor para el control de versiones, aunque amplía sus posibilidades al incorporar una sólida capa de colaboración y “codificación social” [41]. Ofrece un espacio para que los programadores puedan almacenar sus repositorios de Git y promueve la colaboración mediante la transparencia del código. Elementos como las peticiones de integración (pull requests) y el rastreo de problemas (issues) posibilitan que se realicen revisiones y diálogos sobre el código de manera organizada, lo que mejora la administración y la calidad del proyecto [41].

Bitbucket

Bitbucket es una plataforma diseñada para el alojamiento y gestión de repositorios de código, proporcionando un enfoque práctico para el uso de sistemas de control de versiones

como Git. Facilita la colaboración entre desarrolladores mediante herramientas de integración y revisión de código, permitiendo una administración eficiente de las ramas y fusiones del proyecto. Su uso es clave para equipos que buscan mantener un historial de cambios ordenado y seguro dentro del ciclo de vida del desarrollo [42].

Gitlab

GitLab se centra en la gestión integral de repositorios de software, ofreciendo un entorno centralizado para el control de versiones y la colaboración en equipo. Más allá del simple almacenamiento de código, GitLab facilita la administración de proyectos mediante características que apoyan el flujo de trabajo de desarrollo, desde la gestión de usuarios y permisos hasta la revisión de código, consolidándose como una herramienta vital para mantener la coherencia y seguridad en el desarrollo colaborativo [43].

Herramientas de Pruebas y Validación

Herramientas para Pruebas Funcionales

El framework NestJS está diseñado para facilitar las pruebas automatizadas, integrando por defecto un entorno de pruebas robusto [44]. NestJS emplea Jest como su marco de pruebas primordial para las pruebas funcionales, produciendo automáticamente archivos que especifican los componentes. La biblioteca Supertest [44] es la que se usa normalmente para las pruebas de extremo a extremo (E2E), las cuales son esenciales para validar tanto los endpoints de la API como el flujo total de la aplicación. Supertest posibilita emular solicitudes HTTP a los controladores y comprobar las respuestas, lo que garantiza que el backend actúe de la manera prevista [44].

Herramientas para Pruebas No Funcionales

Para las pruebas no funcionales, que analizan características como la eficiencia, el rendimiento y la capacidad de escalar, se utilizan herramientas externas. Para analizar el desempeño de NestJS en condiciones de alta demanda y en ambientes basados en nube, es crucial llevar a cabo pruebas de carga controladas [35]. Para realizar pruebas de rendimiento, simular cargas de usuarios concurrentes y medir los tiempos de respuesta del servidor [45], son frecuentemente usadas herramientas estándar de la industria, como Apache JMe-

ter, en pilas tecnológicas semejantes (por ejemplo MERN, que tiene el mismo entorno Node.js). Este tipo de análisis es esencial para recolectar métricas de rendimiento y garantizar la solidez del backend [46].

Herramientas de Despliegue y Mantenimiento

Docker y Azure App Service

La tecnología de contenedores Docker, ubicada en la plataforma Azure App Service, será la base para implementar los microservicios de Alertify.

Docker es una tecnología de contenedorización que posibilita la agrupación de una aplicación y de sus dependencias (como configuraciones, bibliotecas, etc.) en una unidad estandarizada conocida como contenedor [47]. Para Alertify, esto implica que cada microservicio (Ruteo, Autenticación, Reportes) se agrupará en un contenedor Docker individual. Esta perspectiva asegura que los servicios se lleven a cabo de forma predecible y consistente en cualquier entorno, lo que soluciona los problemas de compatibilidad entre el ambiente de producción y el de desarrollo.

Estos contenedores se implementarán en Azure App Service, un servicio de plataforma como servicio (PaaS) que facilita la gestión y orquestación de aplicaciones alojadas en contenedores [48]. El uso de App Service posibilita abstraerse de la dificultad que conlleva gestionar la infraestructura subyacente (como las máquinas virtuales, las redes, etc.) y brinda características integradas como el escalado automático, el balanceo de carga y la integración/despliegue continuos (CI/CD); estos procesos automatizan la construcción, prueba y despliegue de los contenedores cada vez que se realiza un cambio en el código [49].

Capítulo 2

METODOLOGÍA

El Módulo de Ruteo Seguro busca la optimización de rutas variables haciendo uso de variables de inseguridad ciudadana, por lo cual se optó por un enfoque de investigación aplicada estructurado bajo el marco de trabajo ágil Scrum. En esta sección se describe el proceso de desarrollo organizado en fases iterativas, desde la exploración y planificación hasta la implementación y despliegue, de manera que se asegura la entrega incremental de valor al usuario final.

2.1. Fase Exploratoria

En esta etapa se lleva a cabo la identificación y caracterización de los actores clave que intervienen directa e indirectamente en el sistema, asegurando que las funcionalidades de navegación y seguridad preventiva respondan a necesidades reales.

2.1.1. Stakeholders

Para el desarrollo del módulo es fundamental identificar a las partes interesadas que interactúan directa o indirectamente con el sistema mediante la correcta identificación de sus roles y necesidades permite alinear los requerimientos del algoritmo de ruteo y la interfaz móvil con las expectativas reales de uso y supervisión académica en la Tabla 2.1 se describe a los stakeholders principales.

Tabla 2.1: Partes Interesadas (Stakeholders)

ID	Stakeholder	Persona/Grupo Asignado	Descripción
S1	Ciudadano (Usuario Final)	Ciudadanía de Quito (Jóvenes y adultos)	Usuarios que interactúan con la aplicación móvil para solicitar y visualizar rutas seguras.
S2	Director del Proyecto	Dr. Víctor Vicente Velepucha Bonett	Responsable de supervisar el rigor académico y técnico del proyecto, validando que el motor de ruteo y la arquitectura se cumplan.
S3	Desarrollador	Erick Sebastián Ballas Paguay	Responsable del diseño, construcción e implementación del Módulo de Ruteo Seguro.

2.1.2. Planificación del proyecto

Para el desarrollo módulo, se ha planificado y estructurado una carga de trabajo de 288 horas, la cual se visualiza en la Figura 2.1. Estas horas se disruyen en la investigación preliminar, el diseño arquitectónico, la implementación del código, y la ejecución de pruebas. Cada fase ha sido distribuida de forma progresiva e incremental, garantizando que las actividades técnicas y las tareas organizativas se complementen eficientemente, lo cual permite mitigar riesgos y asegurar la viabilidad técnica del módulo en los plazos establecidos.

En la etapa inicial, el cronograma prioriza una fase de investigación, esencial para establecer la arquitectura que soportará el sistema y la forma en la que se graficara la ruta. Posteriormente, para el cumplimiento del cronograma se inicia con la estructuración de la base de datos, continuando con la implementación de la lógica de negocio en el motor de ruteo y finalizando con la integración de la interfaz móvil para el usuario final. De igual manera a estas actividades de desarrollo, se incorporan las reuniones del marco de trabajo Scrum, tales como las reuniones diarias de seguimiento y las revisiones de Sprint. Asegurando un monitoreo continuo del progreso y permitiendo ajustes tempranos. Por último el proceso finaliza con una fase de validación funcional y pruebas de usabilidad que certifican la calidad y estabilidad del entregable final.

Por otro lado, para la gestión ágil del proyecto, se han definido los roles fundamentales

Actividades Específicas	Mes 1				Mes 2				Mes 3				Mes 4				Mes 5				Mes 6				Horas
	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	
Semanas																									Total
1. Revisión de literatura sobre algoritmos de ruteo y redacción de informe.	5	5																							10
2. Entrevistas con partes interesadas para identificar requerimientos (Ruteo).			5	5																					10
3. Diseño de los prototipos de pantallas (UX/UI) en Figma.					5	5																			10
4. Diseño de arquitectura (Frontend Kotlin, Back end NestJS, BD SQL Server).						5	5																		10
5. Planificación de Sprints (Backlog y refinamiento).						2	2	1	1	1	1														8
6. Desarrollo de Base de Datos Espacial (Red vial, Nodos, Caminos).								12	12	12	12														48
7. Desarrollo Back end: Motor de cálculo de rutas seguras (NestJS).												12	12	12	12										48
8. Desarrollo Frontend: Visualización y solicitud de rutas (Kotlin).																12	12	12	12						48
9. Pruebas funcionales: Validación de algoritmos y cumplimiento de requisitos.																				12	12	12	12		48
10. Pruebas de usabilidad con usuarios y docentes.																				12	12	12	12		48
Eventos Scrum																									
Scrum Diario (15 min/día = 1h/sem)	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	24
Revisión del Sprint (Sprint Review)						2				2				2				1							7
Retrospectiva del Sprint						1				1				1				1					3		7
Total Horas de Trabajo	288																								

Figura 2.1: Cronograma de actividades del proyecto

para el marco de trabajo Scrum. La Tabla 2.2 detalla la asignación de responsabilidades, donde el Dr. Víctor Velepucha asume los roles de dirección y gestión del producto de manera que se garantiza la implementación buena prácticas de Scrum, mientras que el desarrollo técnico es ejecutado por el equipo de estudiantes, con responsabilidades específicas en cada módulo, realizando los incrementos para cada una de las actividades definidas.

Tabla 2.2: Roles y responsabilidades del marco Scrum en el proyecto.

Rol de Scrum	Persona Asignada	Responsabilidad Principal
Scrum Master	Dr. Víctor Vicente Velepucha Bonett	Facilitador del proceso, asegurando la eliminación de impedimentos técnicos y la correcta aplicación de la metodología ágil.
Product Owner	Dr. Víctor Vicente Velepucha Bonett	Responsable de validar que las funcionalidades del motor de ruteo cumplan con los objetivos de seguridad preventiva y aporten valor al usuario final.
Scrum Team	Erick Ballas, Johan Baño, Luis Rocha	Equipo de desarrollo multifuncional encargado de la construcción de los módulos. Específicamente, Erick Ballas lidera la implementación técnica del Módulo de Ruteo Seguro.

2.2. Fase Exploratoria

Esta etapa incluye la configuración del entorno técnico y la especificación minuciosa de los requisitos funcionales del sistema. Se definen las herramientas de desarrollo, se instalan los servicios en la nube y el Product Backlog se establece al traducir los requerimientos de los interesados en historias de usuario que son valoradas y ordenadas por prioridad.

2.2.1. Herramientas y Entorno de Trabajo

A continuación en la Tabla 2.3 se describen brevemente las herramientas y tecnologías seleccionadas para el desarrollo del módulo, asegurando su integración adecuada en la arquitectura de microservicios. La decisión de este stack tecnológico se debe a la necesidad de garantizar una disponibilidad alta y de procesar datos geográficos.

Tabla 2.3: Herramientas y tecnologías utilizadas en el desarrollo del sistema.

Categoría	Herramienta / Tecnología	Justificación Técnica
Gestión y Diseño	Jira	Esta herramienta se utiliza para la gestión de las historias de usuario y seguimiento de los Sprints.
	Figma	Se utiliza para el prototipado de las interfaces de usuario y definición de flujos de navegación.
Desarrollo (IDEs)	Android Studio	Esta herramieta es fundamental para el desarrollo nativo en Kotlin , fundamental para la integración de SDKs de mapas.
	Visual Studio Code	Mediante este editor de código se realiza el desarrollo del backend en NestJS dado a su facilidad de integración y la compatibilidad con múltiples lenguajes [50].
Control de Versiones	Git / GitHub	Para el control de versiones se hace uso de la estrategia <i>Gitflow</i> para la gestión de ramas (develop, main)[40].
Backend & API	NestJS	Mediante este framework se realizara la construcción de microservicios eficientes en el servidor.
Base de Datos	SQL Server	Este es el motor de base de datos relacional seleccionado por su robusto soporte de <i>datos espaciales</i> , esenciales para el algoritmo de ruteo.
Infraestructura	Azure App Service	Servicio PaaS para alojar y escalar [51].

2.2.2. Definición de requerimientos

Para la definición de los requerimientos correspondientes del módulo se estructuró a partir de historias de usuario, en donde primero se redactaron las épicas de usuario dando como prioridad a las funcionalidades de alto nivel. Una vez definidas las épicas, se distribu-

yeron en historias de usuario más detalladas y pequeñas para el Product Backlog con su respectivo desarrollo en los Sprints planificados.

Historias Épicas de usuario

En esta sección se describen las épicas de usuario que abarcan la gestión de datos, el motor de ruteo y la interacción visual con el usuario. En la Tabla 2.4 se detallan con una breve descripción de cada una de ellas y a su vez las historias de usuario asociadas.

Tabla 2.4: Definición de épicas y sus historias de usuario asociadas

Código	Épica	Descripción	Historias Usuario Asociadas
E-01	Gestión del Grafo Vial y Riesgo	Se define las labores de backend y de base de datos que son requeridos, de manera que se permita actualizar los pesos de las aristas (calles) a partir de incidentes relacionados con la seguridad.	HU-04, HU-06
E-02	Motor de Cálculo de Rutas	Organiza la lógica de negocio para implementar los algoritmos de búsqueda que manejan la solicitud y la creación del recorrido.	HU-02, HU-05
E-03	Visualización y Navegación Móvil	Desarrollo de la interfaz del usuario, dando la posibilidad que el usuario interactúe con el mapa, seleccione puntos y visualice las rutas.	HU-01, HU-03

Una vez definidas las épicas de usuario, se presentan su desglose en las tareas específicas para cada historia dando la descripción de cada una, con sus criterios de aceptación y su respectiva prioridad. El detalle mencionado de las historias de usuario se visualizaran en las Tablas 2.5, 2.6, 2.7, 2.8, 2.9 y 2.10.

Tabla 2.5: Historia de Usuario HU-01: Selección de Origen y Destino.

Código	HU-01
Nombre	Selección de Origen y Destino
Prioridad	Alta
Estimación	5 Puntos
Descripción	Como usuario, quiero poder seleccionar mi ubicación actual y un punto de destino mediante una barra de búsqueda o tocando el mapa, para iniciar la solicitud de una ruta.
Criterios de Aceptación	1. Integrar el SDK de mapas permitiendo gestos táctiles. 2. Debe existir un autocompletado de direcciones básicas. 3. El sistema debe capturar las coordenadas (latitud y longitud) de ambos puntos.

Tabla 2.6: Historia de Usuario HU-02: Cálculo de Ruta Segura.

Código	HU-02
Nombre	Cálculo de Ruta Segura
Prioridad	Alta
Estimación	8 Puntos
Descripción	Como usuario, quiero que el sistema calcule una ruta que priorice calles con menor índice de criminalidad, evitando zonas con altos reportes de incidentes.
Criterios de Aceptación	1. El servicio debe recibir el par de coordenadas. 2. El algoritmo debe consultar a la base de datos y utilizar el índice de criminalidad para el peso de las aristas. 3. El sistema debe retornar un archivo con la ruta y el tiempo estimado.

Tabla 2.7: Historia de Usuario HU-03: Visualización de Trayecto.

Código	HU-03
Nombre	Visualización de Trayecto en Mapa
Prioridad	Alta
Estimación	5 Puntos
Descripción	Como usuario, quiero ver dibujada en el mapa la línea de la ruta sugerida para poder seguirla visualmente hacia mi destino.
Criterios de Aceptación	1. La aplicación debe recibir la geometría de la API y dibujarla como una <i>Polyline</i>. 2. La cámara del mapa debe ajustarse para mostrar toda la ruta. 3. Se debe diferenciar visualmente el inicio y el fin con marcadores.

Tabla 2.8: Historia de Usuario HU-04: Ponderación de Riesgo.

Código	HU-04
Nombre	Actualización de Pesos por Incidentes
Prioridad	Media
Estimación	8 Puntos
Descripción	Como sistema, requiero actualizar el costo de una calle cuando se reporta un nuevo incidente, para que las futuras rutas eviten esa zona.
Criterios de Aceptación	1. Debe existir un procedimiento almacenado que intercepte nuevos reportes. 2. Se debe identificar la arista más cercana al incidente y aumentar su coste en el grafo. 3. La actualización debe ser consistente sin bloquear lecturas.

Tabla 2.9: Historia de Usuario HU-05: Recálculo de Ruta por Desvío.

Código	HU-05
Nombre	Recálculo de Ruta por Desvío
Prioridad	Media
Estimación	8 puntos
Descripción	Como usuario, quiero que el sistema detecte si me he desviado de la ruta sugerida y recalcule automáticamente un nuevo trayecto seguro desde mi nueva ubicación hacia el destino original.
Criterios de Aceptación	1. Se debe detectar si la ubicación actual del usuario se aleja de la polilínea de la ruta. 2. Se debe disparar una nueva petición, con las nuevas coordenadas de origen. 3. La transición visual en el mapa debe ser fluida, reemplazando la línea antigua por la nueva sin bloquear la interfaz.

Tabla 2.10: Historia de Usuario HU-06: Carga y Estructuración de Red Vial.

Código	HU-06
Nombre	Carga y Estructuración de Red Vial
Prioridad	Alta
Estimación	5 Puntos
Descripción	Como sistema, debo cargar la cartografía de la ciudad (OpenStreetMap) en la base de datos SQL Server, convirtiendo los datos en una topología de grafos (nodos y aristas) que sea adecuada para el ruteo.
Criterios de Aceptación	1. Importación de datos geográficos de Quito en formato .osm o .shp a tablas de SQL Server. 2. Conversión correcta de coordenadas a tipos de datos Geography o Geometry. 3. Creación de índices espaciales para optimizar las consultas de búsqueda de nodos cercanos.

Escala de Estimación de Complejidad

Se empleó la técnica de Planning Poker, con la secuencia Fibonacci como referencia, para cuantificar el esfuerzo técnico relacionado con cada historia de usuario. Se eligió esta escala por su manera representar la incertidumbre que viene con el desarrollo de software, a medida que la tarea se hace más grande. Esta escala se detalla en la Tabla 2.11.

Tabla 2.11: Escala de Estimación de Complejidad para Historias de Usuario

Puntos de Historia	Descripción de Complejidad
1	Muy Baja
2	Baja
3	Media
5	Alta
8	Muy Alta

Escala de Priorización de Historias de Usuario

Esta respectiva escala permite alinear el desarrollo con los objetivos estratégicos del proyecto garantizando el éxito del mismo, de manera que, se clasifica las historias de usuario en cuatro grados de priorización: Baja, Media, Alta y Muy Alta. Esta escala se referencia en la Tabla 2.12.

Tabla 2.12: Escala de Priorización para Historias de Usuario

Nivel de Prioridad	Descripción
1	Baja
2	Media
3	Alta
4	Muy Alta

Con la definición de la escala de prioridad y de complejidad, se procede a su organización en el Product Backlog, detallando las tareas con mayor impacto para el usuario y la funcionalidad del sistema, asegurando que el desglose esté alineado con su respectiva historia de usuario. A continuación, en la Tabla 2.13 se presenta el desglose de las tareas.

Tabla 2.13: Desglose de tareas por Historia de Usuario

Historia	Tarea	Nombre	Complejidad	Prioridad
HU-01	T-01	Configuración de proyecto e integración inicial de Google Maps SDK.	3	4
HU-01	T-02	Interfaz para búsqueda de direcciones con autocompletado.	3	4
HU-01	T-03	Captura de coordenadas de inicio y fin.	2	4
HU-01	T-04	Validar que las coordenadas estén dentro de Quito.	2	4
HU-02	T-05	Estructuración de controladores para el ruteo.	1	4
HU-02	T-06	Implementación del algoritmo asumiendo la variable de riesgo.	8	4
HU-02	T-07	Obtención de grafos (nodos/aristas) y sus pesos.	5	4
HU-02	T-08	Transformación de la información geográfica y exposición mediante API REST.	3	4
HU-03	T-09	Configuración para consumir el API de ruteo.	2	4
HU-03	T-10	Deducción de la geometría de la ruta y trazado de Polyline en el mapa.	5	4
HU-03	T-11	Implementación de marcadores personalizados para Inicio y Fin.	1	3
HU-03	T-12	Encuadre automático de toda la ruta en pantalla.	2	3

HU-04	T-13	Creación de Stored Procedure para interceptar nuevos reportes.	5	3
HU-04	T-14	Vinculación de incidente con la arista más cercana.	5	3
HU-04	T-15	Automatización de la actualización del campo <code>riesgo</code> .	3	3
HU-05	T-16	Implementación de servicio en segundo plano para monitoreo GPS continuo.	5	4
HU-05	T-17	Detección de desvío de usuario.	5	3
HU-05	T-18	Lógica para disparar automáticamente una nueva petición al backend tras desvío.	3	3
HU-05	T-19	Manejo de transición suave en la UI al reemplazar la ruta antigua.	3	2
HU-06	T-20	Configuración de Azure SQL Database habilitando extensiones geográficas.	2	4
HU-06	T-21	Obtención y limpieza de cartografía de Quito.	3	4
HU-06	T-22	ETL para migrar datos geográficos a tablas <code>Nodos</code> y <code>Aristas</code> .	5	4
HU-06	T-23	Creación de índices geográficos para optimización de consultas.	3	4

2.2.3. Definición de Sprints

Para asegurar un desempeño eficaz y reducir los riesgos, se organizó la planificación operativa en seis ciclos de trabajo con un Sprint 0 como etapa preparatoria. Esta primera iteración busca sentar las bases del proyecto, lo que incluye la preparación de los repositorios, el diseño arquitectónico y la configuración del entorno. Los Sprints 1 a 5 se desarrollan de manera incremental validando la persistencia de datos y el núcleo algorítmico, después se añade la capa visual concluyendo con funcionalidades más complejas como el recálculo de rutas o la ponderación de riesgos. Esta división se detalla en la Tabla 2.14.

Tabla 2.14: Desglose de tareas por Sprint

Sprint	Duración	Objetivo Principal	Tareas Asignadas
0	2 Semanas	Estructuración inicial, realización de diagramas de arquitectura y creación de prototipo.	Fase de planificación.
1	3 Semanas	Configuración de la infraestructura en Azure y estructuración de la Base de Datos Espacial con la red vial de Quito (Requisito indispensable para el ruteo).	De HU-06 (Carga y Estructuración de Red Vial): T-20, T-21, T-22 y T-23. De HU-01 (Selección de Origen y Destino): T-01.
2	4 Semanas	Desarrollo del algoritmo de búsqueda en NestJS y los mecanismos de selección de puntos en la app móvil.	De HU-01 (Selección de Origen y Destino): T-02, T-03 y T-04. De HU-02 (Cálculo de Ruta Segura): T-05, T-06, T-07 y T-08.

Sprint	Duración	Objetivo Principal	Tareas Asignadas
3	3 Semanas	Conexión del cliente móvil con el servidor para visualizar gráficamente la ruta segura calculada.	De HU-03 (Visualización de Trayecto): T-09, T-10, T-11 y T-12.
4	2 Semanas	Implementación de la lógica de base de datos que actualiza los pesos de las calles basándose en incidentes reales.	De HU-04 (Ponderación de Riesgo): T-13, T-14 y T-15.
5	3 Semanas	Desarrollo de capacidades avanzadas para detectar desvíos y solicitar nuevas rutas automáticamente.	De HU-05 (Recálculo por Desvío): T-16, T-17, T-18 y T-19.

2.2.4. Sprint 0

En la etapa inicial del proyecto se crea la infraestructura técnica y los lineamientos arquitectónicos antes de comenzar a desarrollar las funcionalidades de manera progresiva. En esta fase, se pusieron en marcha los repositorios de código, se determinó la estructura del software que respaldará las operaciones de geoprocusamiento, la definición de prototipos y por último la base de datos.

Diagrama de arquitectura

Se adoptó una arquitectura de microservicios para asegurar la escalabilidad y el desacoplamiento funcional del Módulo de Ruteo Seguro en el ecosistema Alertify. Al hacer uso de esta opción se puede separar la carga computacional de los algoritmos de grafos de otros ámbitos del sistema, lo que hace más sencillo el mantenimiento y la escalabilidad autónoma del servicio. La Figura 2.2 ilustra la interacción entre los componentes lógicos del cliente móvil, el servicio de backend y la persistencia de datos.

La arquitectura propuesta organiza el sistema en tres niveles funcionales que interactúan secuencialmente. La aplicación móvil gestiona la interfaz de usuario y la visualización del trayecto sobre el mapa, manteniendo un monitoreo constante de la ubicación geográfi-

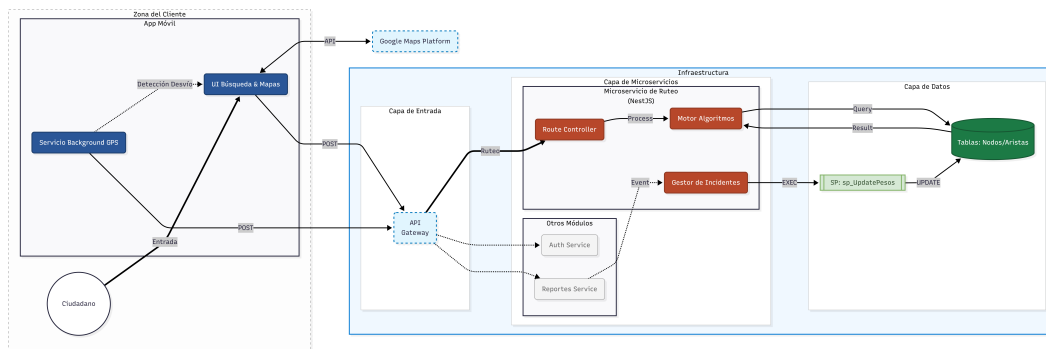


Figura 2.2: Arquitectura de Microservicios del Módulo de Ruteo Seguro en Alertify

ca para trazar la ruta correctamente. La lógica central se basa en recibir las peticiones de navegación y ejecuta los algoritmos de búsqueda sobre el grafo vial. Este módulo tiene la responsabilidad de determinar el camino óptimo tomando en cuenta dos variables: el nivel de riesgo acumulado y la distancia física. Por último, la Capa de Datos guarda el historial de incidentes y la topología de la red vial. Esta estructura posibilita que se hagan consultas espaciales y que los costos de las calles se actualicen en tiempo real, garantizando así que las rutas producidas eludan las áreas con reportes de seguridad recientes.

Prototipos de aplicación móvil

Una vez planteada la arquitectura del módulo, se procedió con el bosquejo de los prototipos de la aplicación móvil. Estos diseños iniciales se enfocan en la experiencia del usuario al interactuar con el mapa, seleccionar puntos de origen y destino, y visualizar la ruta segura sugerida. La Figura 2.3 muestra las pantallas diseñadas en Figma, en donde se visualizan los elementos que conforman la interfaz del aplicativo.

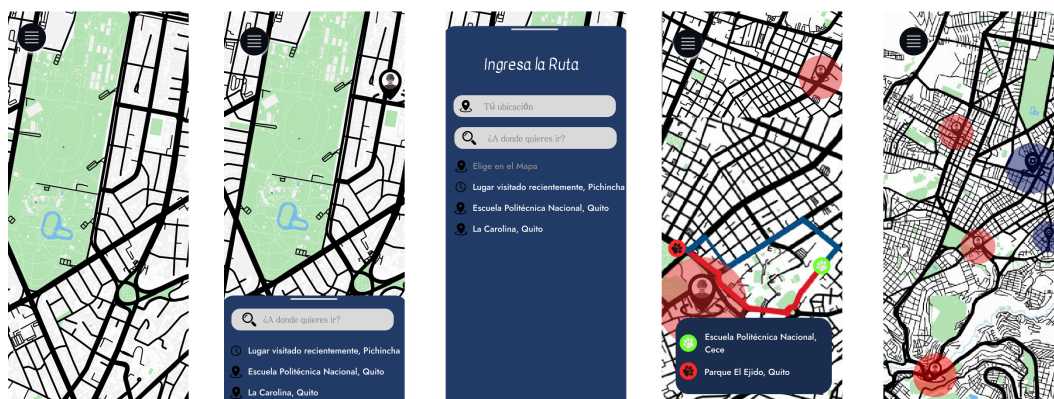


Figura 2.3: Prototipos de la aplicación móvil para el Módulo de Ruteo Seguro

Estructura de la base de datos

El esquema de la base de datos se definió para soportar datos geográficos y permitir consultas eficientes. La Figura 2.4 ilustra las tablas principales que componen la base de datos, destacando las relaciones entre ellas. Los cruces de vías se almacenan en la tabla NODOS como puntos geográficos, a su vez la tabla ARISTAS, que representa los segmentos de calle que enlazan dos nodos esta entidad considera el campo de riesgo como una variable dinámica que se actualiza según los reportes de incidentes, con la cual el algoritmo prioriza en busca de la ruta más segura. Por último, la tabla INCIDENTES almacena los atributos críticos necesarios para el cálculo geométrico (ubicación y gravedad), de esta manera se garantiza que el procedimiento almacenado se ejecute eficazmente.

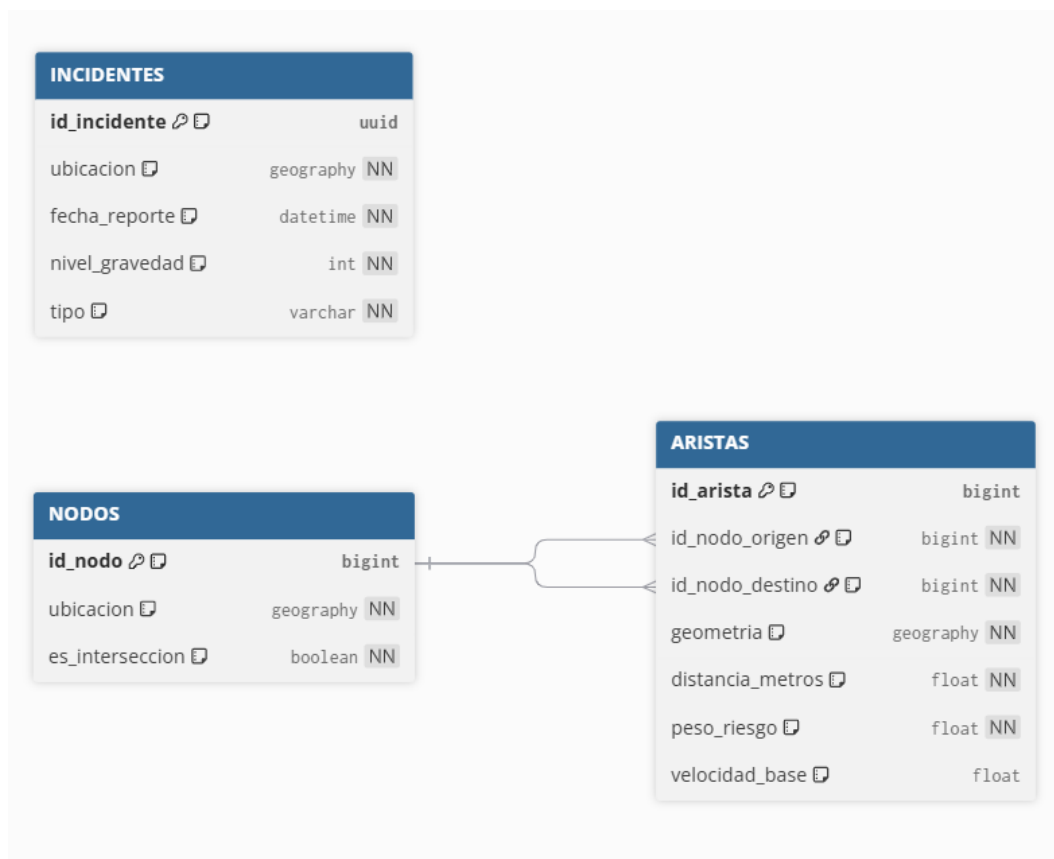


Figura 2.4: Esquema de la base de datos

Configuración de organización en GitHub

Para la distribución y control del código se configuró una organización en GitHub, la cual permite gestionar los repositorios de los diferentes módulos del sistema Alertify facilitando

la colaboración entre los desarrolladores. En la Figura 2.5 se muestra la estructura de la organización y sus respectivos repositorios.

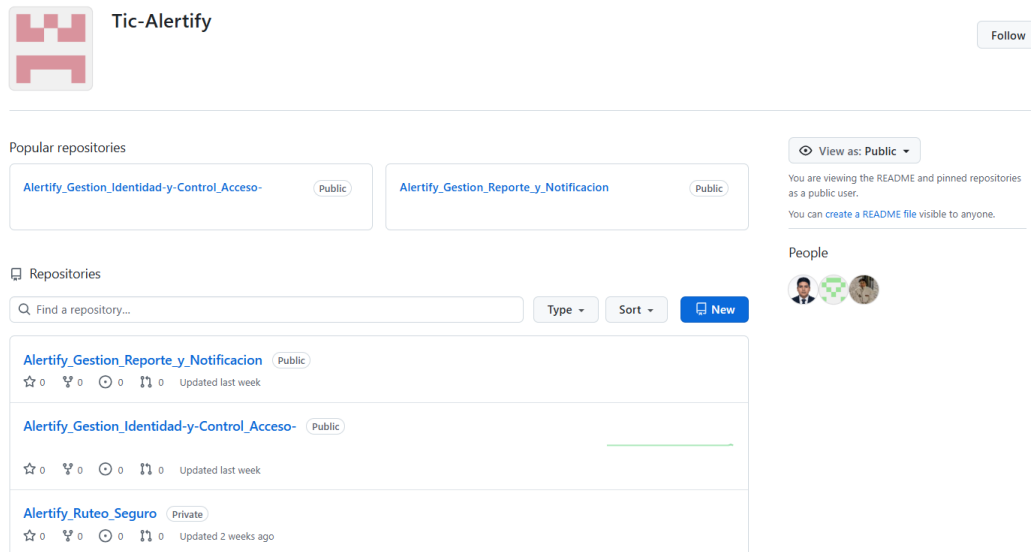


Figura 2.5: Organización en GitHub para el proyecto Alertify

Tablero de gestión en Jira

Para la planificación y supervisión de las tareas, se procedió a la realización del Product Backlog representándolo mediante un tablero en Jira, en el cual se configuró Scrum, donde se transcribieron las historias de usuario y sus respectivas tareas técnicas. En la Figura 2.6, cada ítem de trabajo fue etiquetado con su Épica correspondiente (E-01 Gestión del Grafo, de manera que se garantiza la trazabilidad de los requisitos.

Sprint 0 Review

Se expusieron y se comprobaron los artículos de diseño en la revisión del sprint, en relación con las metas establecidas. Se corroboró que el Diagrama de Arquitectura satisface las necesidades de desacoplamiento para una arquitectura de microservicios, separando adecuadamente la lógica de ruteo del resto de módulos de Alertify. Además, se verificó que el modelo entidad-relación cumpla con la estructura de tablas (nodos y aristas). Por último, la configuración inicial del proyecto estaba completa, por lo que se proclamó el sprint como logrado.

Espacio

MODULO_RUTEO_SEGURO ...

Resumen Cronograma **Backlog** Tablero Calendario Lista Formularios Metas More 5 +

Q Buscar en el ba... EP Filter

☐ **Sprint 1** Añadir fechas (5 actividades) 0 0 0 Iniciar sprint ...

☒	RS-27 T-20 Configuración de Azure SQL Database habilitando extensiones geográfi...	E-01 GESTIÓ...	TAREAS POR ...	-	👤
☒	RS-28 T-21 Obtención y limpieza de cartografía de Quito.	E-01 GESTIÓ...	TAREAS POR ...	-	👤
☒	RS-29 T-22 ETL para migrar datos geográficos a tablas Nodos y Aristas.	E-01 GESTIÓ...	TAREAS POR ...	-	👤
☒	RS-30 T-23 Creación de índices geográficos para optimización de consultas.	E-01 GESTIÓ...	TAREAS POR ...	-	👤
☒	RS-31 T-01 Configuración de proyecto e integración inicial de Google Maps SDK.	E-03 VISUA...	TAREAS POR HA	-	👤

+ Crear

5 de 5 actividades visibles | Estimación: 0 de 0

☐ **Sprint 2** Añadir fechas (7 actividades) 0 0 0 Iniciar sprint ...

☒	RS-32 T-02 Interfaz para búsqueda de direcciones con autocompletado.	E-03 VISUA...	TAREAS POR HA	-	👤
☒	RS-33 T-03 Captura de coordenadas de inicio y fin.	E-03 VISUA...	TAREAS POR HA	-	👤
☒	RS-34 T-04 Validar que las coordenadas estén dentro de Quito.	E-03 VISUA...	TAREAS POR HA	-	👤
☒	RS-35 T-05 Estructuración de controladores para el ruteo.	E-02 MOTOR...	TAREAS POR HA	-	👤
☒	RS-36 T-06 implementación del algoritmo asumiendo la variable de riesgo.	E-02 MOTOR...	TAREAS POR HA	-	👤
☒	RS-37 T-07 Obtención de grafos (nodos/aristas) y sus pesos.	E-02 MOTOR...	TAREAS POR ...	-	👤

Figura 2.6: Tablero de gestión en Jira para el Módulo de Ruteo Seguro

Capítulo 3

RESULTADOS, CONCLUSIONES Y RECOMENDACIONES

3.1. Resultados

Exponer los resultados obtenidos, utilizando para esto el apoyo de tablas, figuras, entre otros.

Un ejemplo de una tabla se presenta en 3.1.

Tabla 3.1: Resultados de las pruebas realizadas

No. Prueba	Resultado	Tiempo [s]
1	10	0.9
2	5	0.5

Un ejemplo de una figura se presenta en la Figura 3.1.

Un ejemplo de una ecuación se presenta en Ecuación 3.1.

$$(a + b)^2 = a^2 + 2ab + b^2 \quad (3.1)$$

Ecuación 3.1. Trinomio Cuadrado Perfecto

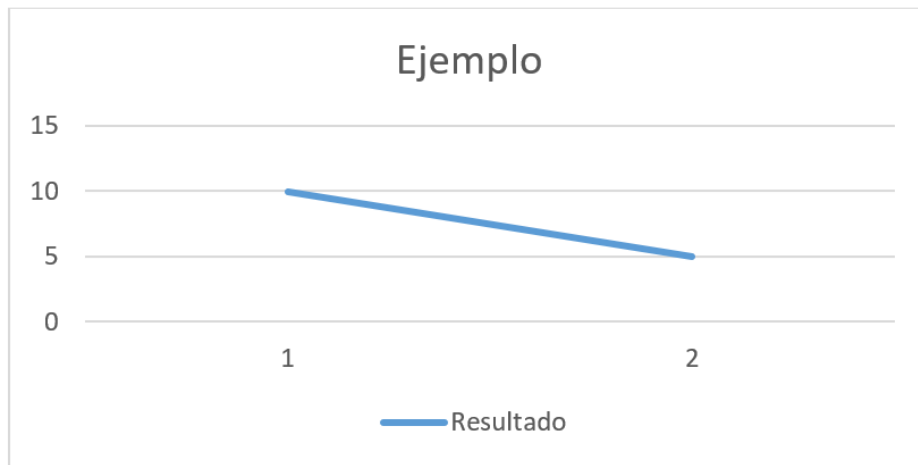


Figura 3.1: Resultados de las pruebas realizadas

3.2. Conclusiones

Presenta lo novedoso del trabajo de integración curricular, así como evaluación del cumplimiento o no de lo propuesto en los objetivos. En el caso en que no se cumpla uno o varios objetivos, y no se logren los resultados esperados, se propone una posible respuesta que explique por qué sucedió esto o las falencias de la planteado.

3.3. Recomendaciones

Indicar las recomendaciones formuladas a partir del desarrollo de este trabajo de integración curricular.

Capítulo 4

REFERENCIAS BIBLIOGRÁFICAS

- [1] G. N. Bravo-Galarraga, K. A. Samaniego-Macas, M. F. Yanéz-USca y L. E. Zuñiga-Ortega, «Seguridad ciudadana en Ecuador: Factores sociales, económicos, confianza institucional y políticas públicas [Citizen security in Ecuador: Social and economic factors, institutional trust and public policies],» *Verdad y Derecho. Revista Arbitrada de Ciencias Jurídicas y Sociales*, vol. 3, n.º especial4, págs. 113-120, 2024.
- [2] R. J. Freire Urresta y R. J. Villegas Mendieta, «Análisis multidimensional sobre la victimización y percepción de inseguridad en 2011 en el Ecuador,» B.S. thesis, 2024.
- [3] A. M. Munizaga, «Aspectos claves acerca del rol de los gobiernos locales en seguridad ciudadana y prevención del delito,» *Conceptos (Fundación Paz Ciudadana)*, vol. 15, 2010.
- [4] L. Dávila Loor, «Puntos críticos de la aplicación de las nuevas tecnologías de información y comunicación (TIC) en la seguridad ciudadana,» Tesis de maestría., Quito: FLACSO sede Ecuador, 2004.
- [5] M. Okmi, L. Y. Por, T. F. Ang, W. Al-Hussein y C. S. Ku, «A systematic review of mobile phone data in crime applications: a coherent taxonomy based on data types and analysis perspectives, challenges, and future research directions,» *Sensors*, vol. 23, n.º 9, pág. 4350, 2023.
- [6] B. M. Espinoza-Marca y S. E. Cieza-Mostacero, «Aplicación Web Móvil en el Control de Incidencias Delictivas,» *Revista Ibérica de Sistemas e Tecnologias de Informação*, n.º E73, págs. 58-74, 2024.

- [7] M. Lobos Vargas, «Efectos de la utilización de aplicaciones móviles de vigilancia sobre la Gobernanza de la Seguridad Urbana a nivel local: el caso de SOSAFE en la Comuna de Santiago de Chile,» 2025.
- [8] J. G. Moya, «Seguridad, amenazas y mecanismos de protección de los dispositivos móviles,» *REVISTA MULTIDISCIPLINARIA DE DESARROLLO AGROPECUARIO, TECNOLÓGICO, EMPRESARIAL Y HUMANISTA.*, vol. 6, n.º 1, págs. 9-9, 2024.
- [9] W. W. Royce, «Managing the development of large software systems: concepts and techniques,» en *Proceedings of the 9th international conference on Software Engineering*, 1987, págs. 328-338.
- [10] A. Moniruzzaman y D. S. A. Hossain, «Comparative Study on Agile software development methodologies,» *arXiv preprint arXiv:1307.3356*, 2013.
- [11] B. Boehm, «A spiral model of software development and enhancement,» *ACM SIGSOFT Software engineering notes*, vol. 11, n.º 4, págs. 14-24, 1986.
- [12] S. Balaji y M. S. Murugaiyan, «Waterfall vs. V-Model vs. Agile: A comparative study on SDLC,» *International Journal of Information Technology and Business Management*, vol. 2, n.º 1, págs. 26-30, 2012.
- [13] N. B. Ruparelia, «Software development lifecycle models,» *ACM SIGSOFT Software Engineering Notes*, vol. 35, n.º 3, págs. 8-13, 2010.
- [14] E.-M. Arvanitou, A. Ampatzoglou, A. Chatzigeorgiou y J. C. Carver, «Software engineering practices for scientific software development: A systematic mapping study,» *Journal of Systems and Software*, vol. 172, pág. 110 848, 2021.
- [15] K. Schwaber y J. Sutherland, «La guía de Scrum,» *Scrumguides. Org*, vol. 1, pág. 21, 2013.
- [16] R. Dixit y B. Bhushan, «Scrum: An Agile software development process and metrics,» *Journal on Today's Ideas-Tomorrow's Technologies*, vol. 7, n.º 1, págs. 73-87, 2019.
- [17] A. A. Manssur Nicola et al., «Aplicación de la metodología Scrum para implementar el esquema gubernamental de seguridad de la información orientado a servicios tecnológicos,» B.S. thesis, 2015.
- [18] G. Blinowski, A. Ojdowska y A. Przybyłek, «Monolithic vs. microservice architecture: A performance and scalability evaluation,» *IEEE access*, vol. 10, págs. 20 357-20 374, 2022.

- [19] B. M. Michelson, «Event-driven architecture overview,» *Patricia Seybold Group*, vol. 2, n.º 12, págs. 10-1571, 2006.
- [20] I. Gorton, *Essential software architecture*. Springer, 2006.
- [21] T. Erl, *SOA Principles of Service Design (paperback)*. Prentice Hall Press, 2016.
- [22] S. Newman, *Building microservices: designing fine-grained systems*. .o'Reilly Media, Inc.", 2021.
- [23] H. Vural, M. Koyuncu y S. Guney, «A systematic literature review on microservices,» en *International Conference on Computational Science and Its Applications*, Springer, 2017, págs. 203-217.
- [24] A. El Akhdar et al., «Exploring the potential of microservices in internet of things: A systematic review of security and prospects,» *Sensors*, vol. 24, n.º 20, pág. 6771, 2024.
- [25] S. Gadge y V. Kotwani, «Microservice architecture: API gateway considerations,» *GlobalLogic Organisations*, vol. 11, 2017.
- [26] S. K. Sharma. «Key Components of Microservice Architecture,» Medium, visitado 11 de ene. de 2026. dirección: <https://medium.com/@sumitkumarsharma/key-components-of-microservice-architecture-7f087940b246>
- [27] V. Sharma y A. K. Tiwari, «A study on user interface and user experience designs and its tools,» *World Journal of Research and Review (WJRR)*, vol. 12, n.º 6, págs. 41-45, 2021.
- [28] D. Sánchez Díez et al., «Open Trivia App: aplicación móvil multiplataforma para juegos basados en preguntas y respuestas,» 2021.
- [29] P. Li, *Jira Software Essentials: Plan, track, and release great applications with Jira Software*. Packt Publishing Ltd, 2018.
- [30] P. S. Uttarwar, R. P. Tidke, D. S. Dandwate y U. J. Tupe, «A literature review on android-a mobile operating system,» *International Research Journal of Engineering and Technology*, vol. 8, n.º 1, págs. 1-6, 2021.
- [31] C. Chaubey y A. Sharma, «The integrated development environment (IDE) for application development: Android studio and its tools,» en *AIP Conference Proceedings*, AIP Publishing LLC, vol. 2427, 2023, pág. 020 087.

- [32] B. P. D. Putranto, R. Saptoto, O. C. Jakaria y W. Andriyani, «A comparative study of java and kotlin for android mobile application development,» en *2020 3rd International Seminar on Research of Information Technology and Intelligent Systems (ISRITI)*, IEEE, 2020, págs. 383-388.
- [33] M. Moskala e I. Wojda, *Android Development with Kotlin*. Packt Publishing Ltd, 2017.
- [34] H. Kesa, «Performance characteristics of current microservice frameworks,» 2021.
- [35] I. Alasmar, *Evaluating the performance of the Node. js frameworks Express, Fastify, and NestJS in modern cloud environments: A comparative analysis of scalability, efficiency, and suitability for high-demand applications in 2025*, 2025.
- [36] A. Meier y M. Kaufmann, *SQL & NoSQL databases*. Springer, 2019.
- [37] R. B. Vellanki, «SQL Server Administration and Maintenance: Best Practices for Modern Environments,» *Journal of Computer Science and Technology Studies*, vol. 7, n.º 6, págs. 839-848, 2025.
- [38] M. Stonebraker y L. A. Rowe, «The design of Postgres,» *ACM Sigmod Record*, vol. 15, n.º 2, págs. 340-355, 1986.
- [39] S. Bradshaw, E. Brazil y K. Chodorow, *MongoDB: the definitive guide: powerful and scalable data storage*. .ºReilly Media, Inc.", 2019.
- [40] J. Loeliger y M. McCullough, *Version Control with Git: Powerful tools and techniques for collaborative software development*. .ºReilly Media, Inc.", 2012.
- [41] L. Dabbish, C. Stuart, J. Tsay y J. Herbsleb, «Social coding in GitHub: transparency and collaboration in an open software repository,» en *Proceedings of the ACM 2012 conference on computer supported cooperative work*, 2012, págs. 1277-1286.
- [42] S. Chakraborty y P. Aithal, «A practical approach to GIT using bitbucket, GitHub and SourceTree,» *International Journal of Applied Engineering and Management Letters (IJAEML)*, vol. 6, n.º 2, págs. 254-263, 2022.
- [43] J. M. Hethey, *GitLab repository management*. Packt Publishing, 2013.
- [44] P. Duc, «DEVELOPING BACK-END OF A WEB APPLICATION WITH NESTJS FRAMEWORK Title of publication Developing back-end of a web application with NestJS framework Case: Integrify Oy's student management system,» 2020.
- [45] S. Aggarwal y J. Verma, «Comparative analysis of MEAN stack and MERN stack.,» *International Journal of Recent Research Aspects*, vol. 5, n.º 1, 2018.

- [46] I. Cardozo Guedes y D. Lauz Infantozzi, «FastRabbit,» 2021.
- [47] V. R. Gudelli, «Containerization technologies: ECR and Docker for microservices architecture,» *International Journal of Innovative Research in Management, Pharmacy and Sciences (IJIRMPs)*, vol. 11, n.º 3, 2023.
- [48] J. N. Acharya y A. C. Suthar, «Docker container orchestration management: A review,» en *International Conference on Intelligent Vision and Computing*, Springer, 2021, págs. 140-153.
- [49] B. Erdenebat, B. Bud, T. Batsuren y T. Kozsik, «Multi-Project Multi-Environment Approach—An Enhancement to Existing DevOps and Continuous Integration and Continuous Deployment Tools,» *Computers*, vol. 12, n.º 12, pág. 254, 2023.
- [50] B. Johnson, *Visual studio code: End-to-end editing and debugging tools for web developers*. John Wiley & Sons, 2019.
- [51] Y. Al-Dhuraibi, F. Paraiso, N. Djarallah y P. Merle, «Elasticity in cloud computing: state of the art and research challenges,» *IEEE Transactions on services computing*, vol. 11, n.º 2, págs. 430-447, 2017.
- [52] H. Kopka, P. W. Daly y S. Rahtz, *Guide to LATEX*. Addison-Wesley Boston, MA, 2004, vol. 4.
- [53] L. Lamport, *LaTeX*. SCompany Cyfronet, 1991.
- [54] F. Mittelbach, M. Goossens, J. Braams, D. Carlisle y C. Rowley, *The LATEX companion*. Addison-Wesley Professional, 2004.
- [55] E. P. Wonohardjo, R. F. Sunaryo e Y. Sudiyono, «A systematic review of SCRUM in software development,» *JOIV: International Journal on Informatics Visualization*, vol. 3, n.º 2, págs. 108-112, 2019.
- [56] W. Zayat y O. Senvar, «Framework study for agile software development via scrum and Kanban,» *International journal of innovation and technology management*, vol. 17, n.º 04, pág. 2 030 002, 2020.
- [57] T. M. Kanade, C. Crasto, S. Patel, H. Wanjare, T. Savale y A. Varghese, «Agile Change Management: Navigating Uncertainty and Complexity,» *International Insurance Law Review*, vol. 33, n.º S4, págs. 127-142, 2025.
- [58] A. Kaur y K. Kaur, «Systematic literature review of mobile application development and testing effort estimation,» *Journal of King Saud University-Computer and Information Sciences*, vol. 34, n.º 2, págs. 1-15, 2022.

- [59] V. G. Lauber, *Designing Complex Software Architecture: Guidelines on Pattern Selection and Microservice Identification Methods*. University of Bamberg Press, 2024, vol. 44.
- [60] A. Kathuria y A. Gupta, «Challenges in android application development: a case study,» *International Journal of computer science and mobile computing*, vol. 4, n.º 5, págs. 294-299, 2015.

ANEXO I

Conjunto de Datos Extensos

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

ANEXO II

Formato de Entrevista

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

ANEXO III

Enlaces

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.